

Data Structures

Python features several different basic integrated structures. In this section we will discuss some of them: lists, tuples, sets and dictionaries. (eventually generator and comprehension)

1. Lists

One of the most flexible structures in Python is the list. It is a grouping of values. The creation of a list is done by writing the values by separating them with a comma and surrounding them by square brackets ([and]).

```
x = ["Amine", "Sarah", "Zohra", "Adel"]  
print(x)
```

Output: ['Amine', 'Sarah', 'Zohra', 'Adel']

The content of a list is not necessarily text:

```
y = [1, 2, 3, 4, 5]  
print(y)
```

Output: [1, 2, 3, 4, 5]

It is even possible to include elements of different types in a list:

```
z = ["Piketty", "Thomas", 1971]  
print(z)
```

Output: ['Piketty', 'Thomas', 1971]

A list can contain another list:

```
tweets = ["aaa", "bbb"]  
followers = ["Anne", "Bob", "Irma", "John"]  
compte = [tweets, followers]  
print(compte)
```

Output: [['aaa', 'bbb'], ['Anne', 'Bob', 'Irma', 'John']]

1.1. Extraction of the Elements

Access to the elements is made thanks to its indexation (be careful, the index of the first element is 0):

```
print(x[0]) # The first element of x
```

Output: 'Amine'

```
print(x[1]) # The second element of x
```

Output: 'Sarah'

Access to an element can also be done by starting from the end, by putting the minus sign (-) in front of the index:

```
print(x[-1]) # The last element of x
```

Output: 'Adel'

```
print(x[-2]) # The penultimate element of x
```

Output: 'Zohra'

Splitting a list so as to obtain a subset of the list is done with the colon (:):

```
print(x[1:2]) # The first and second elements of x
```

Output: ['Sarah']

```
print(x[2:]) # From the second element (not included) to the end of x
```

Output: ['Zohra', 'Adel']

```
print(x[: -2]) # From the first to the penultimate (not included)
```

Output: ['Amine', 'Sarah']

Remark: The extraction from a list using the brackets returns a list. When extracting items from the list using the brackets, it is possible to add a third argument, the step :

```
print(x[::2]) # Every other element
```

Output: ['Amine', 'Zohra']

Access to nested lists is done by using the brackets several times:

```
tweets = ["aaa", "bbb"]
```

```
followers = ["Anne", "Bob", "Irma", "John"]
```

```
conuts = [tweets, followers]
```

```
res = conuts[1][3] # The 4th item of the 2nd item on the list counts
```

The number of elements in a list is obtained with the function len() :

```
print(len(conuts))
```

Output: 2

```
print(len(conuts[1]))
```

Output: 4

1.2. Modification

Lists are mutable, i.e., their content can be modified once the object has been created.

1.2.1. Replacement

To modify an element in a list, the indexes can be used:

```
x = [1, 3, 5, 6, 9]
```

```
x[3] = 7 # Replacing the 4th element
```

```
print(x)
```

Output: [1, 3, 5, 7, 9]

1.2.2. Insertion

To add an element to a list, the method insert() can be used:

```
x = [1, 3, 5, 7, 9]
x.insert(2, 4) # Inserting the value 4 at the 3rd position
print(x)
Output: [1, 3, 4, 5, 7, 9]
```

1.2.3 Adding Elements

To add items to a list, the method `append()` can be used:

```
x.append(11) # Add value 11 at the end of the list
print(x)
Output: [1, 3, 4, 5, 7, 9, 11]
```

It is also possible to use the `extend()` method, to concatenate lists:

```
y = [13, 15]
x.extend(y)
print(x)
Output: [1, 3, 4, 5, 7, 9, 11, 13, 15]
```

1.2.4 Deleting Elements

To removing an item from a list, the method `remove()` can be used:

```
x.remove(3) # Remove the fourth element
print(x)
Output: [1, 4, 5, 7, 9, 11, 13, 15]
```

The `del` command can also be used:

```
x = [1, 3, 5, 6, 9]
del x[3] # Remove the fourth element
print(x)
Output: [1, 3, 5, 9]
```

1.2.5. Multiple assignments

Several values can be modified at the same time:

```
x = [1, 3, 5, 6, 10]
x[3:5] = [7, 9] # Replaces 4th and 5th values
print(x)
output: [1, 3, 5, 7, 9]
```

The modification can increase the size of the list:

```
x = [1, 2, 3, 4, 5]
x[2:3] = ['a', 'b', 'c', 'd'] # Replaces the 3rd value
print(x)
Output: [1, 2, 'a', 'b', 'c', 'd', 4, 5]
```

Several values can be deleted at the same time:

```
x = [1, 2, 3, 4, 5]
x[3:5] = [] # Removes the 4th and 5th values
print(x)
```

Output: [1, 2, 3]

1.3. Verifying if a Value is Present

By using the operator `in`, it is possible to test the belonging of an object to a list:

```
x = [1, 2, 3, 4, 5]
print(1 in x)
```

Output: True

1.4. Copy of List

Be careful, copying a list is not trivial in Python. Let's take an example.

```
x = [1, 2, 3]
y = x
```

Let's modify the first element of `y`, and look at the content of `y` and `x`:

```
y[0] = 0
print(y)
output: [0, 2, 3]
```

```
print(x)
Output: [0, 2, 3]
```

As can be seen, using the equal sign simply created a reference and not a copy.

To copy a list, there are several ways to do so. Among them, the use of the `list()` function:

```
x = [1, 2, 3]
y = list(x)
y[0] = 0
print("x : ", x)
Output: x : [1, 2, 3]
```

```
print("y : ", y)
Output: y : [0, 2, 3]
```

It can be noted that when a splitting is done, a new object is created, not a reference:

```
x = [1, 2, 3, 4]
y = x[:2]
y[0] = 0
print("x : ", x)
Output: x : [1, 2, 3, 4]
```

```
print("y : ", y)
Output: y : [0, 2]
```

1.5. Sorting

To sort the objects in the list (without creating a new one), Python offers the method `sort()` :

```
x = [2, 1, 4, 3]
x.sort()
print(x)
```

Output: [1, 2, 3, 4]

It also works with text values, sorting in alphabetical order:

```
x = ["c", "b", "a", "a"]
x.sort()
print(x)
```

Output: ['a', 'a', 'b', 'c']

It is possible to provide the `sort()` method with arguments. Among these arguments, there is one, `key`, which provides a function for sorting. This function must return a value for each object in the list, on which the sorting will be performed. For example, with the `len()` function, which, when applied to text, returns the number of characters:

```
x = ["aa", "a", "aaaaa", "aa"]
x.sort(key=len)
print(x)
```

Output: ['a', 'aa', 'aa', 'aaaaa']

In []: